

Processz

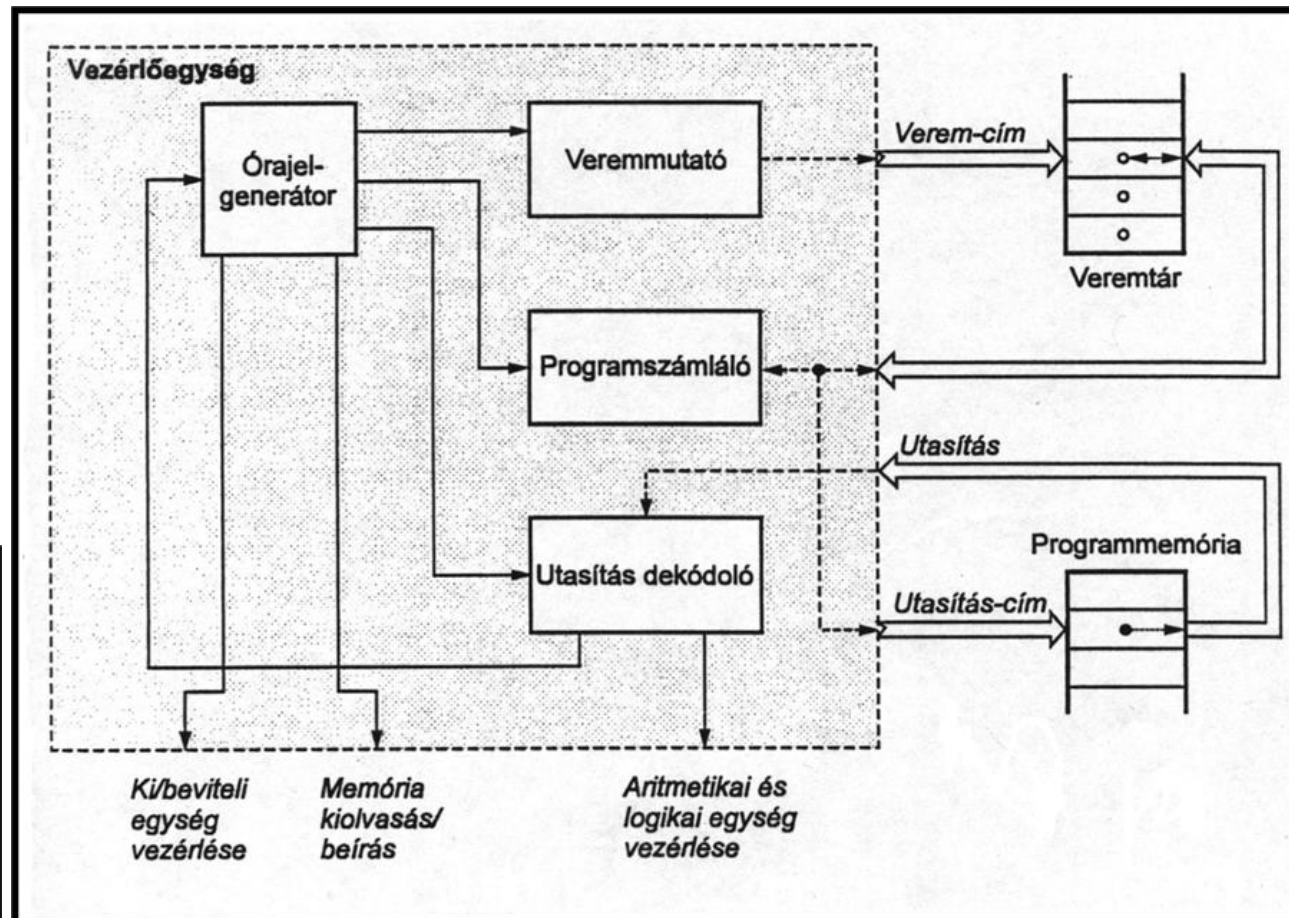
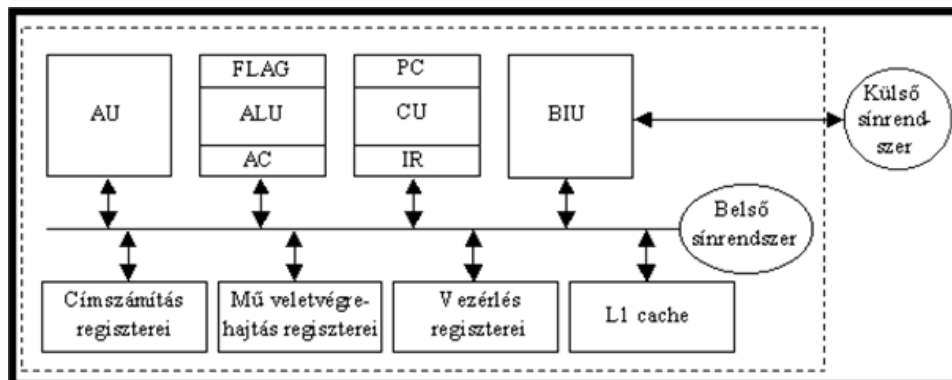
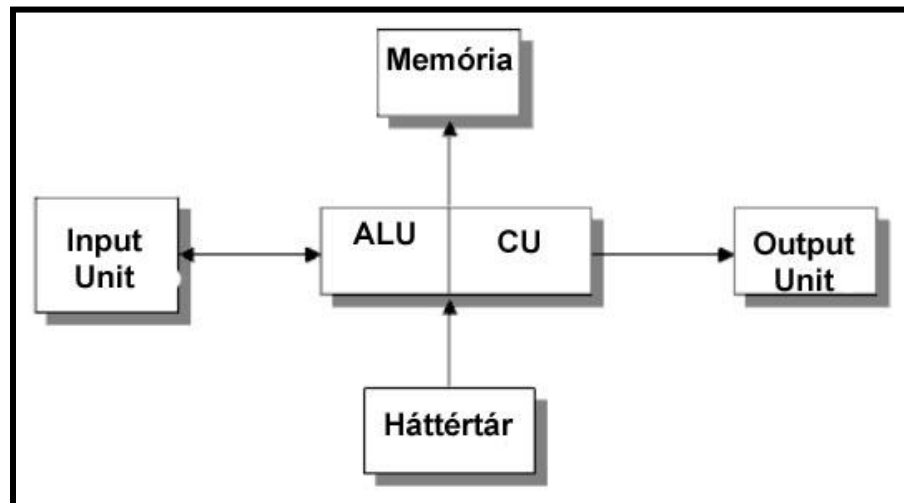
Az előadás anyaga

Dr. Vadász Dénes: Operációs rendszerek
jegyzete alapján készült el

Témakörök

- A processz és a processz kontextus fogalmak.
- Kontextus váltás (processz – rendszer – processz kontextus).
- Processz kontroll, rendszerhívásai.
- Processz állapotok, állapotátmenetek, futási mód.

Mikroszámítógép elvi felépítése



CPU által végrehajtott instrukciófolyam

„Hogyan is működik a számítógép?”

- A CPU veszi a soron következő *gépi instrukciót* és azt végrehajtja.
- A PC (Program Counter) *regiszter* tartja nyilván, melyik a soron következő *gépi instrukció*.
- Egy-egy instrukció végrehajtása után a PC regiszter „automatikusan” felveszi a soron következő instrukció címét.

CPU által végrehajtott instrukciófolyam

- A memóriában *tároltak az instrukciók*.
- A CPU által *végrehajtott instrukciófolyam* egy *instrukciósorozat*, amely nem mindig egyezik a *tárolási sorozattal* (pl. ugróutasítások).
- Így azt is mondhatjuk, hogy az *instrukciófolyamat* az *strukturálatlan*.
- De...

CPU által végrehajtott instrukciófolyam

De ...a HW ad egy strukturáltságot

- CPU által végrehajtott *instrukciófolyam* egy része *user mode* fut, más részei pedig *kernel mode-ban*.
- *User mode* és *kernel mode* végrehajtott instrukciófolyam között un. **szakaszok** vannak, - köztük a bizonyos *futási mód* váltás és a *visszaállítás* jelenség.
- Mi a *trap*? (kivételes esemény, amely a *user mode* a *kernel mode* váltást idéz elő, pl: fájlműveletet vagy hálózati hozzáférést kér.

Az instrukciófolyam másik strukturálása

„Logikailag” szakaszoljuk az *instrukciófolyamot*.

- Egy *adott program* futásához tartozó instrukciófolyam a kezdetétől a végéig - ez a *szakasz*.
- Egy-egy ilyen szakaszt nevezhetünk *munkának* (job), *folyamatnak* (processz) - ez a *szakaszolás*.
- Ezt a *szakaszolást* az OS végzi, *tartja nyilván, menedzseli*.

Processz

Processz fogalma

Processz fogalma

Az OS kulcsfontosságú fogalma a *folyamat* fogalom.

A köznyelv és más szakterületek is használják pl. tevékenységek sorozatának végrehajtását értjük, aminek van kezdete és vége.

Az OS esetén elsősorban a *folyamat* (process) kifejezést használjuk.

Kapcsolatos kifejezések a *munka* (job), a *feladat* (task), a *fonál*, vagy *szál* (thread) kifejezések is.

Processz fogalma – első megfogalmazás

A **processz** fogalma:

A processz – egy végrehajtási példánya egy párhuzamosságot nem tartalmazó végrehajtható programnak.

A processz egy futó program-példány.

Tehát, megkülönböztetjük a *processz* fogalmat a *végrehajtható program* (executable program) fogalomtól.

A processz más entitás (létezik valami), mint a program!

Processz fogalma

- Processz != végrehajtható program!
- Pongyolán: processzben fut a program.

Mi a processz?

A végrehajtás alatt levő program.

Program egy algoritmust megvalósító utasítások sorozata, függetlenül attól, hogy azok *magas szintű nyelven*, akár *bináris gépi kódban* van ábrázolva.

Processz fogalma – kitérő

Hogyan is készül el egy program?

- Egy program egy *editorral* készült *forrás* program.
- Lefordítva a *forrás* programot *tárgyprogram* kapunk.
- *Tárgyprogramot* egy *szerkesztő* eszköz segítségével *futásra kész állapotba* hozhatunk.
- Az eredmény a *futásra kész, végrehajtható* program.
- A programnak *helyfoglalása* van, *méretük* van, *statikusak*.

Processz fogalma

Tehát, a program a *végrehajtandó utasítások sorozata*.

- A *processz* egy *végrehajtás alatt lévő program*.
- A processz egy *szála* az, ami éppen fut egy CPU-n, és nem maga a processz.
- Minden *processzhez* tartozik legalább *egy* szál, ami elindulásakor elkezd futtatni a *program main metódusát*.

Processz fogalma - életciklusa

Egy processz születik.

- Születése után egy *végrehajtható program* betöltődik a *memóriába* és *fut*.
- A processz *vetélkedik* az erőforrásokért, *"él", időt használ*, viselkedése *dinamikus*.
- Végül a processz *megszűnik* (exitál, terminálódik).

Tehát, a processzek *születnek, vetélkednek, együttműködnek, kommunikálnak, végül megszűnnek*.

Processz fogalma - életciklusa

Multiprogramozott rendszerben *több processz* létezhet *egyidőben*, mint ahány *processzor* van (Intel Core i9-10900K 10-Core 3.7GHz LGA1200 Box (EN) Processzor: 10 mag, 20 szál).

Mi az *OS feladata*?

- biztosítson a *processzek számára „logikai processzort”*,
- *ütemezze számukra a valódi processzor idejét*,
- „kapcsolja” számukra a *valódi processzort*,
- gondoskodjon a *„logikai processzorok” állapotának megtartásáról*.

Processz kontextus – fogalma

Folyamat tartalom (process context) *fogalom*

Process context – folyamat környezet fordítás – az nem jó, mert az *process environment*.

A *folyamat tartalom* vagy *folyamat szövegekörnyezet* használható kifejezések.

Ebből is következhet, hogy használhatjuk a *processz kontextust*.

Processz kontextus

A folyamat kontextus definíciója:

Adatstruktúrákba rendezve minden olyan információ, ami a folyamat futásához szükséges.

Más fogalmazásban:

Minden olyan információ, ami a rendszer számára szükséges.

.

Processz kontextus

Mi a processz *kontextus tartalma*?

- a program *kódszegmensei* (instrukciók sorozatai),
- a program *adatszekciói* (inicializált adatszekciók - kezdőérték beállítás), konstans szekciók stb.),
- a processz *veremtárai* (stack), az argumentumátadáshoz,
- *regiszter állapotok*,
- a processz *menedzselési információi*: *azonosítási, erőforrási és állapotokat leíró, hozzáférési információk*.

Processz kontextus - egyik szemléletmód

Hogyan szemlélhető a *processz kontextus*? – többféle módon:

a.) *hardver kontextus*:

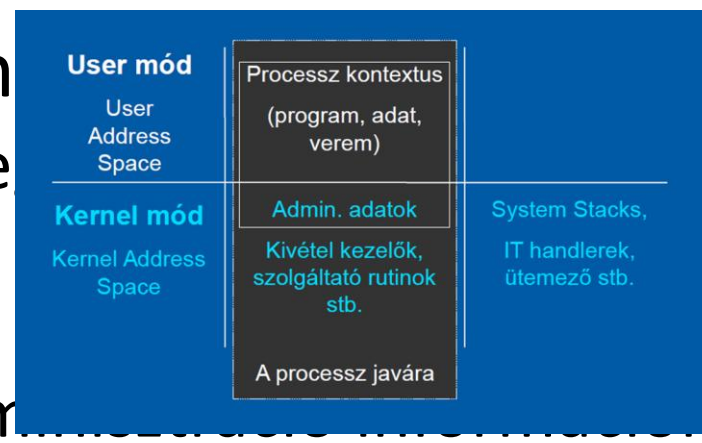
- *HW regiszterek* (a regiszterek *pillanatnyi értékei* - mint *menedzselési információk*) - (CS, DS, ES, SS, a szegmens mutatók - ezek kódszegmensek: adatszegmens, az extra szegmens és a verem szegmens),
- *MMU regiszterek* stb.

b.) *szoftver kontextus* (pl.: kód és adatszegmensek, menedzselési inf.)

Processz kontextus - másik szemléletmód

Címtartomány

- *User level context*: (kód, adat), ami a *User Address Space* címtartományban van (pl.: kódszövegek),
- *System level context*:
 - *statikus rész* (pl. menedzselési, adminisztratív információk)
 - *dinamikus rész* (lebegő) (pl. a regiszterállapotok).



Miért nevezzük *lebegőnek* (volatile)? Mert...

regiszterek értékeit maguk a regiszterek hordozzák, időnként azonban le vannak mentve valamilyen *verembe*.

Regiszterek - ismételés

A regiszterek a processzorban található kis-méretű tárolók.

Általános célú regiszterek

- AX – akkumulátor: aritmetikai műveletek,
- BX – bázis regiszter: tömbindexelés,
- CX – számláló (count) regiszter: ciklusszervezés, ismétlések számolása,
- DX – adat (data) regiszter: I/O műveletek, aritmetikai műveletek.

Szegmens regiszterek

- CS – kód szegmens: futtatandó utasítások tárolása (maga a program),
- DS – adat szegmens: globális változók, konstansok tárolása,
- SS – verem szegmens: lokális változók tárolása, visszatérési cím,
- ES – extra szegmens: tetszőlegesen felhasználható terület (pl. heap).

Regiszterek - ismételés

Pointer (index) regiszterek

- IP – utasításszámláló, az éppen következő assembly utasításra mutat,
- SI – forrás (source) index, sztringműveletek kiinduló címe,
- DI – cél (destination) index, sztringműveletek érkezési címe,
- SP – verem mutató (stack pointer), a verem aktuális tetejét mutatja,
- BP – bázis pointer, bázis-index címzéshez (tömbindexelés).

https://www.inf.u-szeged.hu/~vargalg/oktatas/assembly_18t/tananyag/regiszterek.html

Regiszterek - ismételés

Speciális regiszter (SR = FLAGS)

Státusz regiszter, minden bitje egy nevesített jelzőérték (true/false).

O - előjeles átvitel (overflow)

D - sztringműveletek iránya

I - interruptok engedélyezése

T - csapda (debug) eng.

S - eredménye előjele (sign)

C - eredménye átvitele (carry)

Z - eredménye nulla-e (zero)

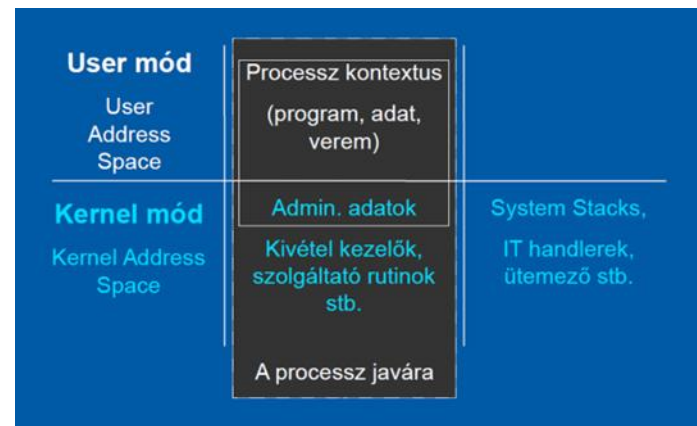
A - félszavas átvitel (BCD)

P - eredmény páros/páratlan

Futási mód - kontextus - címtartomány

Címtartomány fogalmai:

- A *processzek* által használt ún. *virtuális címek*.
- Egy *processz* szerint a *virtuális címtartománya nagy* és ehhez *memória rekeszeket* kell biztosítani – ezt a *hardver* és az *OS kernel* együtt adja.



Futási mód - kontextus - címtartomány

A virtuális címeket leképezi fizikai címekre ehhez biztosítani kell, hogy a fizikai rekeszben ott is legyen az érvényes (valid) információ (instrukció vagy adat).

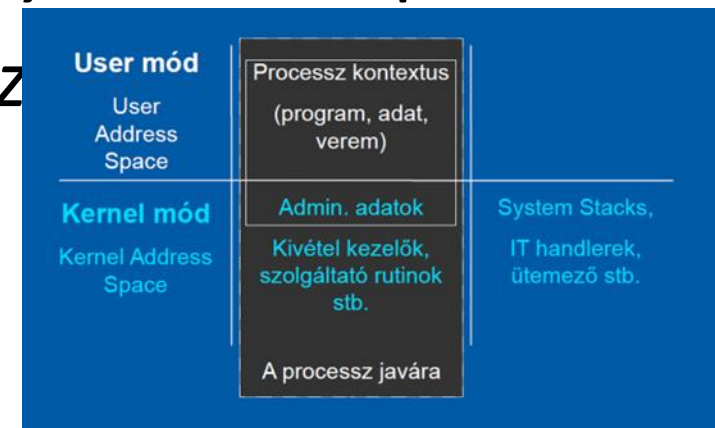
Virtuális címtartomány: azok a címek, amelyekre a program hivatkozni tud.

Fizikai címtartomány: azok a címek, amelyek tényleges memória cellát címeznek.

Futási mód - kontextus - címtartomány

A címtartomány:

- Egy része a *felhasználói címtartomány*. Processzenként *szétválasztott memóriaterületet* címez.
- Másik része *kernel szintű címtartomány*. Minden processz ugyanazt *a kernel memóriarészt* címez



CPU futási módok - processz kontextus

a.) A CPU futhat egy **processz kontextusán** (kontextusában)

- *Felhasználói módban*: a program kódját futtatja.
- *Kernel módban*: *system call* rutint, *kivételkezelőt* futtat.

b.) A CPU futhat **rendszer kontextusban**

- *Kernel módban*: IT kezelést végez, tevékenysége nem köthető egy processzhez (sem).

Processz kontextus váltások

Kontextus váltások

a.) *Processz kontextusról - rendszer kontextusra* való váltás (pl. IT esetén),

Teendők:

- processz kontextusának lementése (valójában csak a dinamikus kontextust kell lementeni),
- a rendszer kontextus betöltése.

Processz kontextus váltások

b.) *Rendszer kontextusról - processz kontextusra* váltás (pl. visszatérés IT-ből)

Teendők:

- *a processz kontextus visszatöltése.*

Lehetséges gyorsítás: több regiszter készlet a CPU-ban.

(HW támogatás – pl. HyperThreading)

Processz kontextus váltások

c) *Processz-processz kontextus váltás* (Process Context Switch):

A CPU "kapcsolása" két processz között (P1, P2).

- A P1 "elveszti" a CPU-t, *lementődik a kontextusának megfelelő része,*
- A P2 „betöltődik” a *kontextusa* és megkapja a CPU-t.

Teendők:

- *Lekapcsolt (P1) processz kontextusának lementése,*
- *Felkapcsolt (P2) processz kontextusának betöltése.*

Processzek listázása a rendszeren

Pl.: processzek listázására:

- ps – futó folyamatok kiírása

ALAKJA: ps [opciók]

```
$ ps
```

PID	TTY	TIME	CMD
22833	pts/1	00:00:00	bash
22859	pts/1	00:00:00	ps

Ezek csak az "aktuális" processzek, ami az aktuális TTY-hez van rendelve.

- PID - Process ID - egyedi azonosító
- TTY – melyik terminálról adta ki, melyik terminál tartozik hozzá (TeleTYpe) (pts - pseudo terminal slave)
- TIME - az elhasznált CPU idő
- CMD - a "parancs" neve

Processzek listázása a rendszeren

Pl.: processzek listázása:
a - all - minden (aminek van terminálja (tty))

```
$ ps a
```

PID	TTY	STAT	TIME	COMMAND
22833	pts/1	Ss	0:00	-bash
22983	pts/2	Ss+	0:00	-bash
23006	pts/1	R+	0:00	ps a

Ez "minden" processz, de valójában nem minden.

Csak, amihez van terminál rendelve.

Új oszlop:

STAT - a processz állapota

(S: sleeping, alszik; R: running, fut;
Z: zombie)

Processzek listázása a rendszeren

```
$ ps auxu
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.0	10780	824	?	Ss	Sep19	0:00	init [2]
root	2	0.0	0.0	0	0	?	S	Sep19	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	S	Sep19	0:00	[ksoftirqd/0]
root	5	0.0	0.0	0	0	?	S<	Sep19	0:00	[kworker/0:0H]
root	6	0.0	0.0	0	0	?	S	Sep19	0:00	[kworker/u:0]
root	7	0.0	0.0	0	0	?	S<	Sep19	0:00	[kworker/u:0H]
...										
k	16382	0.0	0.0	4316	616	?	Ss	00:04	0:00	sh -c uxterm -font -misc-fixed-medium-r-normal--20-200-75-75-c-100-iso10646-1 -vb -b 0 -fg green3 -bg black
k	16383	1.0	0.0	70792	5552	?	S	00:04	0:00	xterm -class UXTerm -title uxterm -u8 -font -misc-fixed-medium-r-normal--20-200-75-75-c-100-iso10646-1 -vb -b 0 -fg green3 -bg
k	16387	2.0	0.0	25068	5420	pts/3	Ss	00:04	0:00	bash
k	16407	0.0	0.0	19256	1312	pts/3	R+	00:04	0:00	ps auxu

Processzek list

```
$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.0	10780	824	?	Ss	Sep19	0:00	init [2]
root	2	0.0	0.0	0	0	?	S	Sep19	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	S	Sep19	0:00	[ksoftirqd/0]

Ezen már valójában ott lesz minden, a *"rendszer" processzek* is:

- USER - a processz tulajdonosának neve.
- PID - processz ID.
- %CPU - a *processz CPU használata %-os arányban*.
- %MEM - a processz aktuális memóriahasználata az össz. memóriához viszonyítva.
- VSZ - virtuális memória mérete, ami a processzhez tartozik.

Processzek listázása a rendszeren

- RSS - a rezidens memória mérete (ami ténylegesen használt memória).

```
$ ps auxu
```

- TTY – fentebb.

- STAT – fentebb.

- START - mikor indult a processz.

- COMMAND - a processz "parancs" neve.

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.0	10780	824	?	Ss	Sep19	0:00	init [2]
root	2	0.0	0.0	0	0	?	S	Sep19	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	S	Sep19	0:00	[ksoftirqd/0]

Processzek listázása

```
$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.0	10780	824	?	Ss	Sep19	0:00	init [2]
root	2	0.0	0.0	0	0	?	S	Sep19	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	S	Sep19	0:00	[ksoftirqd/0]

- `a` - all - minden (aminek van terminálja (tty)),
- `w` - wide (ne vágja le a sor végét ha hosszú, hanem tördelje új sorba),
- `x` - listázza *azokat is, amikhez nem tartozik terminál (tty)*, szokás mondani, hogy "*rendszer folyamatok*" (de persze nem feltétlen rendszer ezek mind),
- `u` - felhasználó orientált kiíratási formátum.
- `less` – szűrő oldalakra bontva vizsgálja - `$ ps aux | less`

Processzek listázása a rendszeren

Mit lehet meg ezen kívül kinyerni a *processz kontextusokból*?

List Open Files, az összes nyitott file-t.

\$ lsof

COMMAND	PID	TID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
bash	16426		k	cwd	DIR	253,6	45056	2138240	/home/k
bash	16426		k	rtd	DIR	8,1	4096	128	/
bash	16426		k	txt	REG	8,1	975488	239	/bin/bash
bash	16426		k	mem	REG	8,1	52160	2118037	/lib/x86_64-linux-gnu/libnss_files-2.17.so
bash	16426		k	mem	REG	8,1	43592	2117838	/lib/x86_64-linux-gnu/libnss_nis-2.17.so
bash	16426		k	mem	REG	8,1	89104	2118028	/lib/x86_64-linux-gnu/libnsl-2.17.so
bash	16426		k	mem	REG	8,1	31632	2118030	/lib/x86_64-linux-gnu/libnss_compat-2.17.so
bash	16426		k	mem	REG	253,4	3877168	118619	/usr/lib/locale/locale-archive
bash	16426		k	mem	REG	8,1	1737136	2117949	/lib/x86_64-linux-gnu/libc-2.17.so
bash	16426		k	mem	REG	8,1	14664	2118024	/lib/x86_64-linux-gnu/libdl-2.17.so
bash	16426		k	mem	REG	8,1	167952	2117974	/lib/x86_64-linux-gnu/libtinfo.so.5.9
bash	16426		k	mem	REG	8,1	145160	2117932	/lib/x86_64-linux-gnu/ld-2.17.so
bash	16426		k	mem	REG	253,4	26258	6900479	/usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
bash	16426		k	0u	CHR	136,4	0t0	7	/dev/pts/4
bash	16426		k	1u	CHR	136,4	0t0	7	/dev/pts/4
bash	16426		k	2u	CHR	136,4	0t0	7	/dev/pts/4
bash	16426		k	255u	CHR	136,4	0t0	7	/dev/pts/4

Processzek listázása a rendszeren

\$ lsof

COMMAND	PID	TID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
bash	16426		k	cwd	DIR	253,6	45056	2138240	/home/k
bash	16426		k	rtd	DIR	8,1	4096	128	/
bash	16426		k	txt	REG	8,1	975488	239	/bin/bash
bash	16426		k	mem	REG	8,1	52160	2118037	/lib/x86_64-linux-gnu/libnss_files-2.17.so
bash	16426		k	mem	REG	8,1	43592	2117838	/lib/x86_64-linux-gnu/libnss_nis-2.17.so
bash	16426		k	mem	REG	8,1	89104	2118028	/lib/x86_64-linux-gnu/libnsl-2.17.so
bash	16426		k	mem	REG	8,1	31632	2118030	/lib/x86_64-linux-gnu/libnss_compat-2.17.so
bash	16426		k	mem	REG	253,4	3877168	118619	/usr/lib/locale/locale-archive
bash	16426		k	mem	REG	8,1	1737136	2117949	/lib/x86_64-linux-gnu/libc-2.17.so
bash	16426		k	mem	REG	8,1	14664	2118024	/lib/x86_64-linux-gnu/libdl-2.17.so
bash	16426		k	mem	REG	8,1	167952	2117974	/lib/x86_64-linux-gnu/libtinfo.so.5.9
bash	16426		k	mem	REG	8,1	145160	2117932	/lib/x86_64-linux-gnu/ld-2.17.so
bash	16426		k	mem	REG	253,4	26258	6900479	/usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
bash	16426		k	0u	CHR	136,4	0t0	7	/dev/pts/4
bash	16426		k	1u	CHR	136,4	0t0	7	/dev/pts/4
bash	16426		k	2u	CHR	136,4	0t0	7	/dev/pts/4
bash	16426		k	255u	CHR	136,4	0t0	7	/dev/pts/4

- COMMAND - a processzhez tartozó parancs
- PID - process ID.
- TID - thread ID.
- USER - felhasználó, akinek a tulajdona a processz.
- FD - file descriptor azonosító (ahol értelmezhető.)
- TYPE - milyen típusú file (REG - regular/hagyományos, CHR - character device, DIR - directory).

Processzek listázása a rendszeren

- **DEVICE** - *major, minor azonosítója*, a fájl tároló eszköz - ahol található a *file* vagy éppen ha egy *dev file* van megnyitva, akkor annak az *azonosítója*.
- **SIZE/OFF** - méret/offset, ahol értelmezhető
- **NODE** - inode száma
- **NAME** - file neve

\$ lsdf

COMMAND	PID	TID	USER	FD	TYPE	DEVICE	SIZE/OFF	NODE	NAME
bash	16426		k	cwd	DIR	253,6	45056	2138240	/home/k
bash	16426		k	rtd	DIR	8,1	4096	128	/
bash	16426		k	txt	REG	8,1	975488	239	/bin/bash
bash	16426		k	mem	REG	8,1	52160	2118037	/lib/x86_64-linux-gnu/libnss_files-2.17.so
bash	16426		k	mem	REG	8,1	43592	2117838	/lib/x86_64-linux-gnu/libnss_nis-2.17.so
bash	16426		k	mem	REG	8,1	89104	2118028	/lib/x86_64-linux-gnu/libnsl-2.17.so
bash	16426		k	mem	REG	8,1	31632	2118030	/lib/x86_64-linux-gnu/libnss_compat-2.17.so
bash	16426		k	mem	REG	253,4	3877168	118619	/usr/lib/locale/locale-archive
bash	16426		k	mem	REG	8,1	1737136	2117949	/lib/x86_64-linux-gnu/libc-2.17.so
bash	16426		k	mem	REG	8,1	14664	2118024	/lib/x86_64-linux-gnu/libdl-2.17.so
bash	16426		k	mem	REG	8,1	167952	2117974	/lib/x86_64-linux-gnu/libtinfo.so.5.9
bash	16426		k	mem	REG	8,1	145160	2117932	/lib/x86_64-linux-gnu/ld-2.17.so
bash	16426		k	mem	REG	253,4	26258	6900479	/usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
bash	16426		k	0u	CHR	136,4	0t0	7	/dev/pts/4
bash	16426		k	1u	CHR	136,4	0t0	7	/dev/pts/4
bash	16426		k	2u	CHR	136,4	0t0	7	/dev/pts/4
bash	16426		k	255u	CHR	136,4	0t0	7	/dev/pts/4

Processzek listázása a rendszeren

`top`

- Valós időben írja ki a *futó processzeket*, a CPU időt - csökkenően rendezve.
- Információt ad a *processzor*, a *memória használatáról*, a *processzek azonosítóiról*, prioritásairól, a processzt indító felhasználóról, a futási időről, a processz-indító parancsról.
- A fejlécben összesítés szerepel.

Processzek listázása a rendszeren

Hálózati kapcsolatok: (ezek is mind processzhez tartoznak)

Recv-Q / Send-Q - Fogadó oldali / Küldő oldali várakozó sorban mennyi adat van

```
$ netstat -anp
```

```
Active Internet connections (servers and established)
```

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
PID/Program name					
tcp	0	0	192.168.1.11:41465	193.6.5.4:22	ESTABLISHED
16354/ssh					
tcp	0	0	0.0.0.0:22	0.0.0.0:*	LISTEN

```
....
```

```
Active UNIX domain sockets (servers and established)
```

Proto	RefCnt	Flags	Type	State	I-Node	PID/Program name	Path
unix	2	[ACC]	STREAM	LISTENING	4897	-	/tmp/.winbindd/pipe
unix	2	[ACC]	STREAM	LISTENING	4899	-	/var/run/samba/winbindd_privileged/pipe
unix	2	[ACC]	STREAM	LISTENING	8228	-	/tmp/.X11-unix/X0

Processzek listázása a rendszeren

`ps` - aktuális processzekről készít jelentést.

Opciói:

- `ps -A` - összes processz kiválasztása,
- `ps -Al` - egyes processzek paramétereit,
- `ps -AlFH` -szálakat mutatja,
- `ps ax` -a szerver összes processzeit,

Processzek listázása a rendszeren

- `ps aux` – *tulajdonos, az időpontot, amikor elindult, valamint különféle erőforrás használati információkat,*
- `ps -ejH` – processz fa kinyomtatása,
- `ps tree` - a futó processzek listáját fa elrendezésben,
- `ps -p 1286 -o comm=` - egy adott PID neve jelenjen meg,
- `ps -auxf | sort -nr -k 3 | head -5` – 5 legtöbb CPU memóriát fogyasztó folyamat azonosítása.

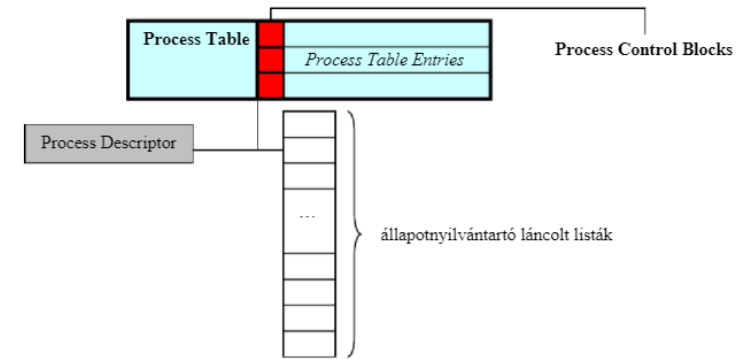
Processz kontextus adatstruktúrái

A processzek kezeléséhez az OS magja *adatstruktúrákba rendezve* tartja nyilván minden *processz kontextusát*.

Az adatstruktúrák elemei :

- *processz tábla* (Process Table),
- *processz tábla bejegyzés* (Process Table Entry),
- *processz leíró* (Process Descriptor),
- *az állapotokat nyilvántartó láncolt listák.*

Processz kontextus adatstruktúrái

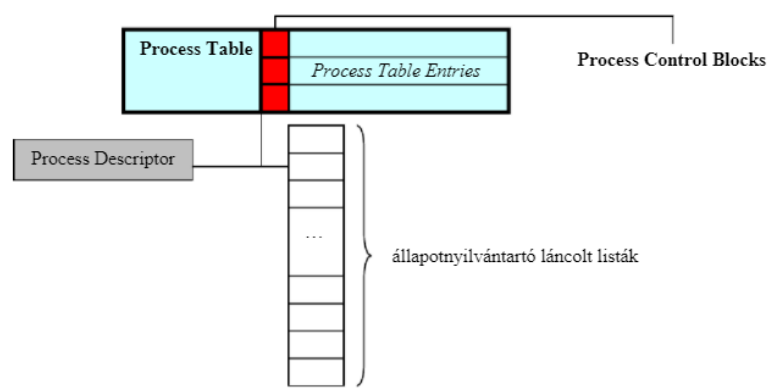


A processz tábla - (Process Table) - **kiinduló pont.**

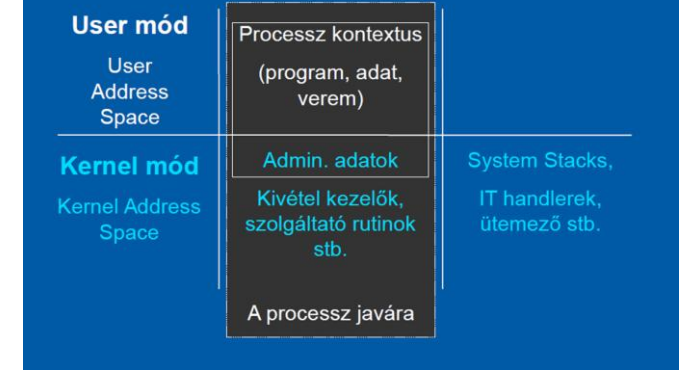
A kernel által kezelt, adott méretű táblázat, annyi bejegyzése van, amennyi az élő processzek száma.

Egy-egy bejegyzése egy-egy Process Table Entry (PTE) vagy Processz Vezérlő Blokk (Process Control Block, PCB).

- Minden processznek van benne saját bejegyzése.
- A tábla méretét *rendszergeneráláskor* szabhatják meg.



us adatstruktúrái



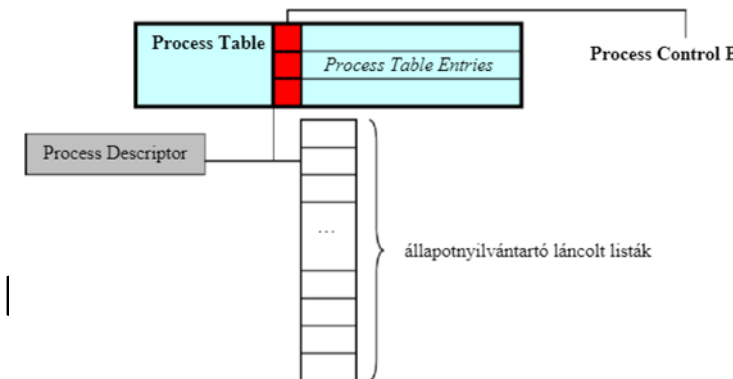
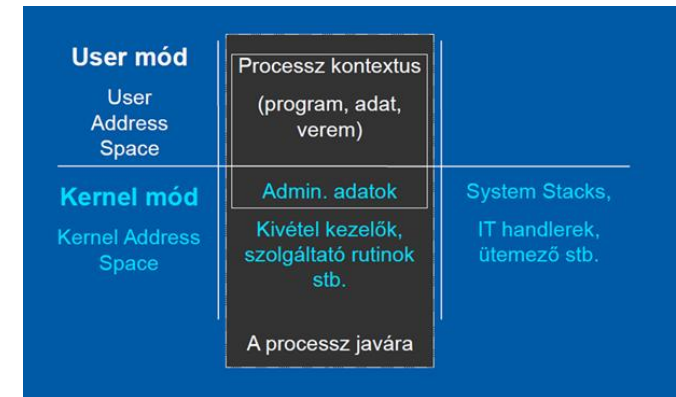
A **processz tábla** a *kernel címtartományában* van.

- Új bejegyzése (processz belépési pont) keletkezik egy processz *kreálásakor* és *megszűnésekor*.
- A processzről "**statikus**" **adatokat** tartalmaz: olyanokat, melyek a *processz létezését jelzik*, de még nem (feltétlenül) *elegendőek a processz futásához*.

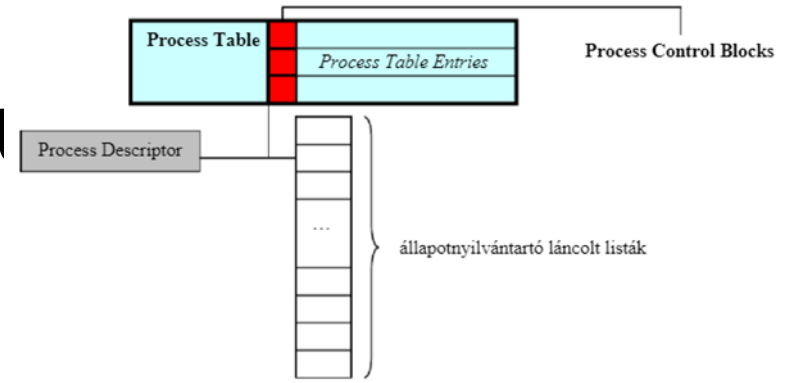
Processz kontextus adatstruktúrái

Process Table Entry (kernel címtartományban)

- a processz tábla egy-egy bejegyzése,
- a processz *kreáláskor* keletkezik,
- "*statikus*" adatokat tartalmaz:
 - azonosítók (pid, pname, statikus kvóták, attl)
 - kód és adatszegmens pointererek,
 - vermek hossza stb.



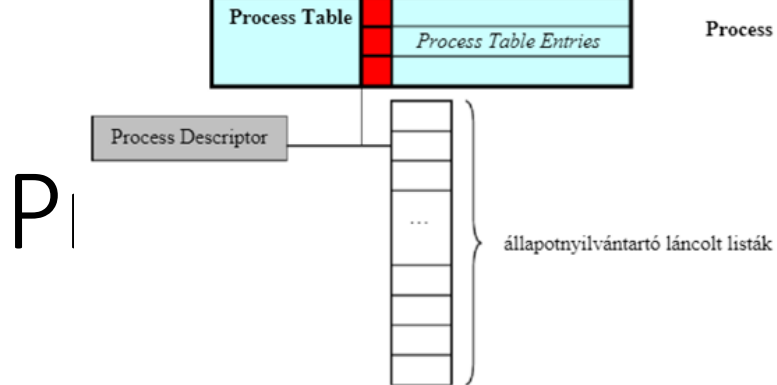
Processz kontextus adatstruktúra



Process Table Entry :

- a)** *menedzselési információkat és azonosítási információk (pid, pname, attribútumok) tartalmaz - ill. egyéb „statikus” infókat.*
- b)** *a mutatók a kontextus további részeire (kód és adatszegmenseire, stb.)*

Bejegyzés tartalmaz *előre/hátra mutató pointereket* is, melyek a *láncolt listájának kezeléséhez* szükségesek.



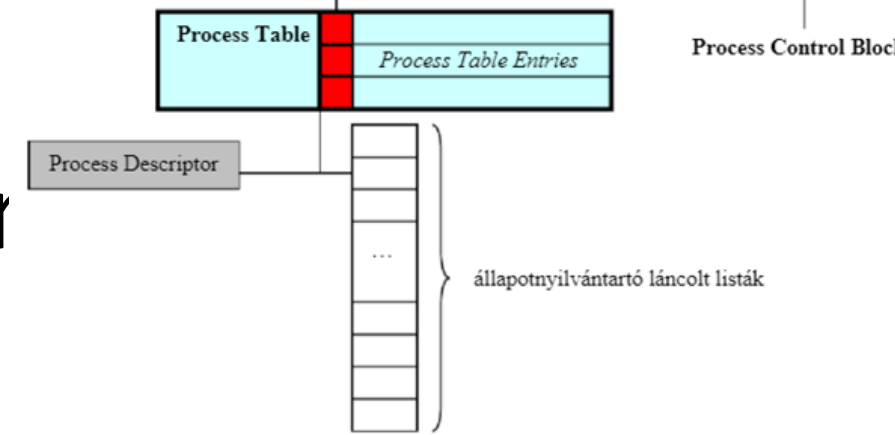
adatstruktúra



Processz leíró (Process Descriptor) - kernel által kezelt struktúra, a *kernel címtartományában*.

- a processz *futtathatóvá válásakor* keletkezik,
- "*dinamikus*" adatokat tartalmaz (a processz futásához is kell):
 - volatile kontextus (regisztertartalmak: PC stb.),
 - ütemezési, állapot információk,
 - veremek pointerei stb.

Processz kontextus adatstruktúr



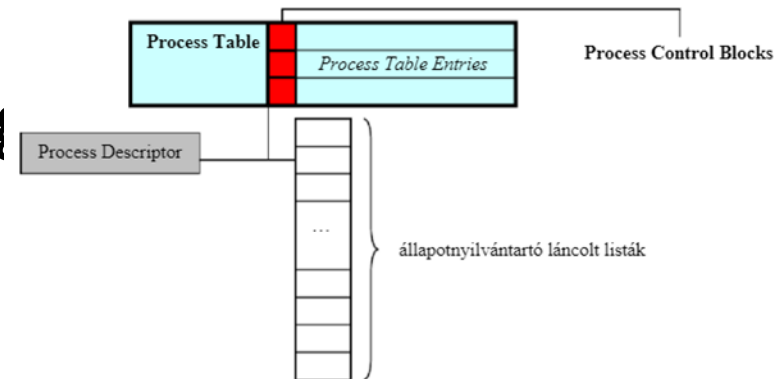
Processz vezérlő blokk (Process Control Block)

- *processz azonosítási információi* (pid, ppid, pname),
- *processz számlázási információi* (CPU használat stb.),
- *időkiosztással kapcsolatos információk*,
- *erőforrás használati határértékek* (limitek, kvóták),
- *mutatók a kontextus további részeire* stb.,
- *next/prev mutató pointer*ek láncolt listákhoz.

(Más elnevezések is lehetnek)

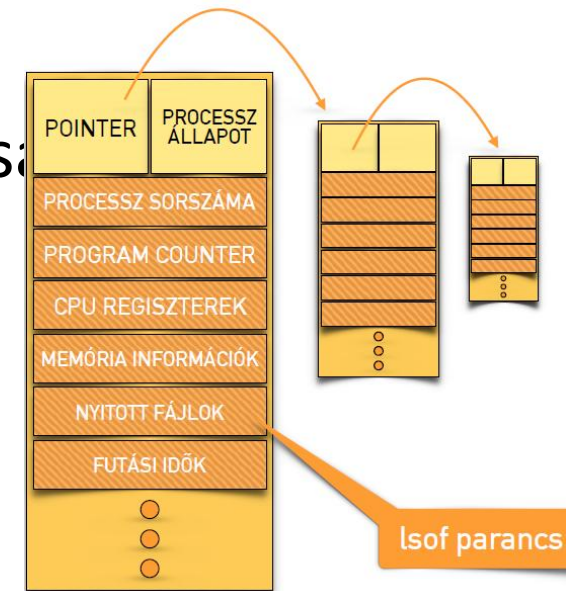
Processz kontextus adatstruktúrái

- *processz állapot* - processz aktuális állapotát írja le (futó, futáskész, blokkolt stb.)
- *program counter* - a processzek végrehajtása során nyilván kell tartani, hogy az éppen hol tart
- *CPU regiszterek* - a processzor regiszterjeit tárolja.



Processz kontextus adatstruktúrái

- *nyitott fájlok* - a rendszernek nyilván kell tartani, megvalósítható legyen az a *zárolási mechanizmus*, amely megakadályozza, hogy egy fájlt több processz is írni próbáljon – erre szolgál a **lsof** (LiSt Open Files) parancs.
- *futási idők* - a `time` parancs egy program futásá-



Forrás: KoczkaF

Processz vezérlése

Processz vezérlése

Processzek vezérlése (Process Control, Process Management)

A processzek

- *születnek,*
- *élnek, valamilyen feladatot végeznek,*
 - *versenyeznek erőforrásokért* (pl. CPU, RAM, diszkek),
 - *osztoznak erőforrásokon* (pl. kernel kódon),
 - *kommunikálnak, együttműködnek,*
 - *szinkronizáció köztük.*
- *exitálnak* (megszűnnek).

Processz vezérlése

Születés

- Általában *processzt csak processz kreálhat*.
- Így: *szülő - gyermek* kapcsolat lehet közöttük.
- Van *összülő processz* (pl. „init”).
- A létrehozás *adminisztrációs munkával* jár.

Processz vezérlése

Kreáció műveletei

- A kernel elkészíti a PT Entry-t,
- memóriát biztosít a *kód és adatszegmenseknek* (programkód, konstansok, init változók);
- *memóriát biztosít a vermeknek,*
- elkészíti a *regiszter kontextust* (a PC/IP értéket kap!),
- *futtatható állapotba kerül a processz.*

Processz rendszerhívásai – fork

Fork(): új processz létrehozása.

- A `fork()` rendszerhívással lehet *gyermek processzt* létrehozni.
- A létrehozás után a *gyermek- és a szülő-processz* egymással párhuzamosan fognak futni.
- A `fork()` rendszerhívás *C nyelvben* egy `integer` típusú értékkel tér vissza, ami alapján meg lehet különböztetni, hogy melyik a *szülő-* ill. melyik a *gyermek-processz*.

Processz rendszerhívásai – fork()

- A *gyermek*-processzben a *visszatérési érték*: 0, míg
- A *szülő*-processzben a *visszatérési érték* egyenlő a *gyermek*-processz *pid-jével*.
- *Negatív visszatérési érték* a rendszerhívás sikertelenségét jelzi.

Processz rendszerhívások: *create()* , *fork()*, *exec()*, *load()*, *run()*, *system()* etc.

Rendszerhívás osztályok – nem teljes

Processz menedzsment

- create, terminate process
- load, execute
- end, abort
- get, set process attributes
- wait for time, sleep
- wait for event, signal event
- send & receive messages
- allocate free memories

Informálódó, beállító

- get, set time, date etc.

Fájlok, jegyzékek

- create, delete files
- open, close files
- read, write, reposition
- get, set file attributes

Eszköz manipulációk

- request, release devices
- read, write, reposition
- get, set device attributes
- logically attach, detach devices

Fork() – példa

*Egy processzt a **fork()** rendszerfüggvénnyel hozhatunk létre!*

```
1  #include <stdio.h>
2
3  int main(int argc, char *argv[])
4  {
5
6      int ppid, pid, gpid;
7      ppid = getpid(); /* szülője azonosítóját a getppid függvénnyel kérdezhető le */
8      pid = fork(); /* Gyermekek születik */
9      if(pid == 0)
10     {
11         /* Itt a gyermek fut */
12         gpid = getpid(); /*a saját PID lekérdezésére a getpid függvény*/
13         exit(0); /* Gyermekek megszüntetése */
14     }
15     /* itt a szülő fut */
16     exit(1); /* Szülő megszüntetése */
17
18 }
```

Processz rendszerhívásai - exec

EXEC

- Az `exec` rendszerhívással lehet egy *programot végrehajtani*. Ilyen pl.: az `execve()` vagy `execl` rendszerhívás.
- Az `exec` lényege, hogy a processz PID-je nem változik, tehát a hívó processz „helyére” töltődik be a meghívott program.

Processz rendszerhívásai - exec

Program futtatás (exec)

Az exec utasítás különböző *végződésekkel* rendelkezhet:

l – a paraméterátadás *listán keresztül* történik,

v – a paraméterátadás *tömbön keresztül* történik,

e – változik a környezeti változó,

p – figyelembe veszi a PATH változót.

Függvények	Paraméter	Keresési útvonal	Környezet
execl	lista	./	marad
execv	tömb	./	marad
execvp	lista	PATH	marad
execvp	tömb	PATH	marad
execle	lista	./	változik
execve	tömb	./	változik

Processz vezérlése - befejeződés

Processz befejeződés

- *normális termináció*: `exit()` / `return` rendszerhívás család egyik tagjának meghívása, vagy a *main* függvény befejeződése (!)
- *kierőszakolt termináció* (más processzből): az `abort()` (vagy `kill()`) rendszerhívással (közvetetten szignállal).

Processz vezérlése

Processz befejeződés

- A processz befejeződése közben
 - elengedi a foglalt erőforrásokat,
 - szignáláz a szülőjének, hogy *exital*.

Ettől kezdve ő egy un. „*zombi*” processz (már nem fut, de még létezik)

Még *adminisztrációs munka van* a törlődésig.

Processz vezérlése - életciklusa

A processzek életük során:

- *versenyeznek erőforrásokért:*
 - CPU-ért: ütemezés, processz állapotok.
 - Egyéb erőforrásokért: *erőforrás kezelés, holtpont probléma.*
- *kommunikálnak:* IPC mechanizmusok.
- *együttműködnek: szinkronizáció megoldása, versenyhelyzet (konkurencia) probléma*

Processz állapotok

Processz állapotok

Az OS szempontjából nézve:

- több processz (futó program) létezik a rendszerben,
- ezeknek szükségük van a CPU-ra, tehát - versengenek érte.
- Az alapján, hogy a CPU-ért versenyzésben éppen ki hol tart különböző *állapotokat* különböztetünk meg.

Egy állapotának megváltozása, állapotátmenet egy másik állapotba.

Processz állapotok

Mi *OS feladata?*:

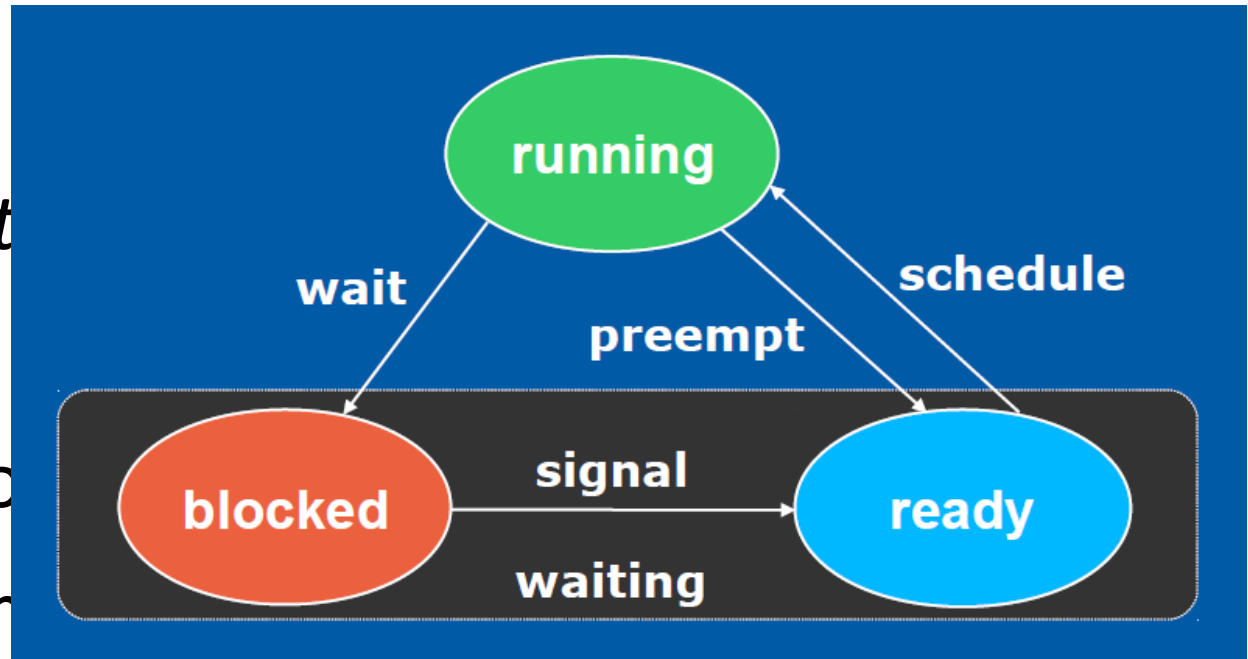
- *nyomon kell követni a processz állapotokat,*
- *állapotváltozásokat,*
- *nyilván kell tartania az állapotokat.*

Sőt, vezérelni, lebonyolítani kell az állapotátmeneteket.

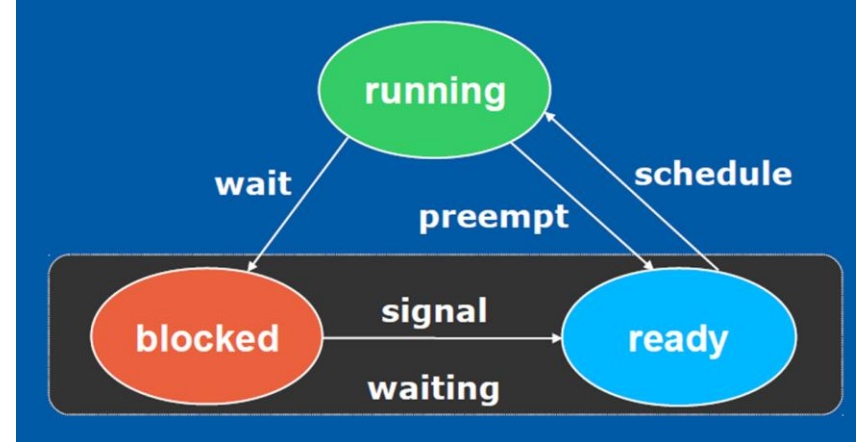
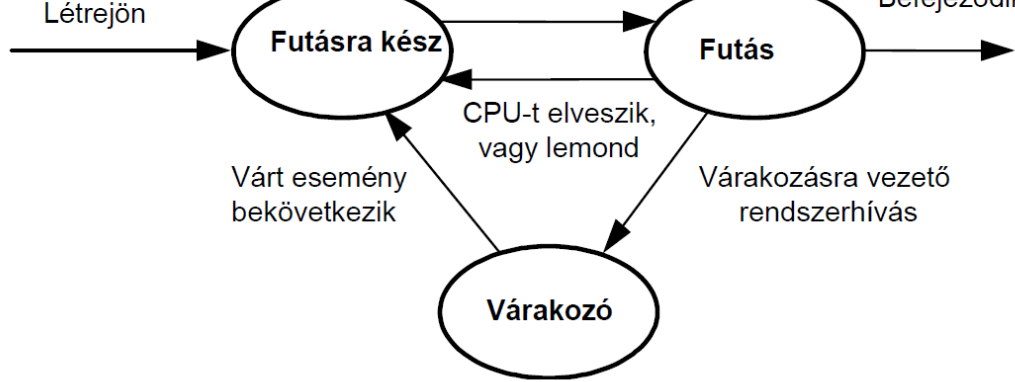
Processz állapotok, állapotátmenetek

Az állapotokat, állapotátmeneteket a processz modellben vizsgáljuk:

- *Ellipszisek az állapotokat jelölik*
- *Mi válthatja ki az állapotátmenetet?*



Válasz: a Process Context Switch jelenség



A legalapvetőbb állapotok:

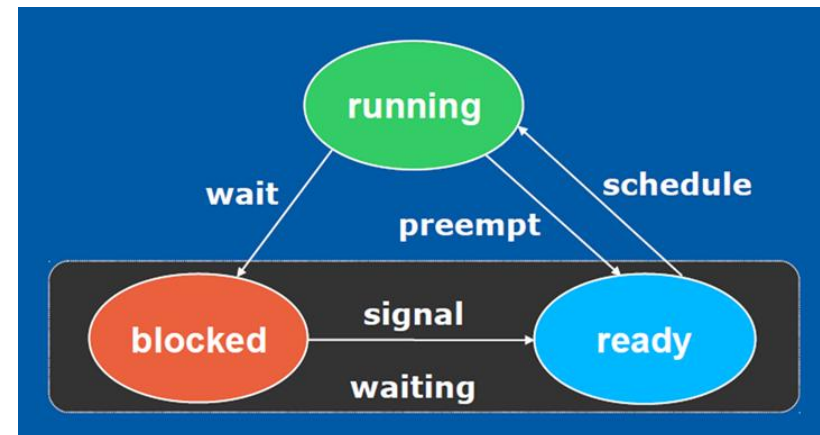
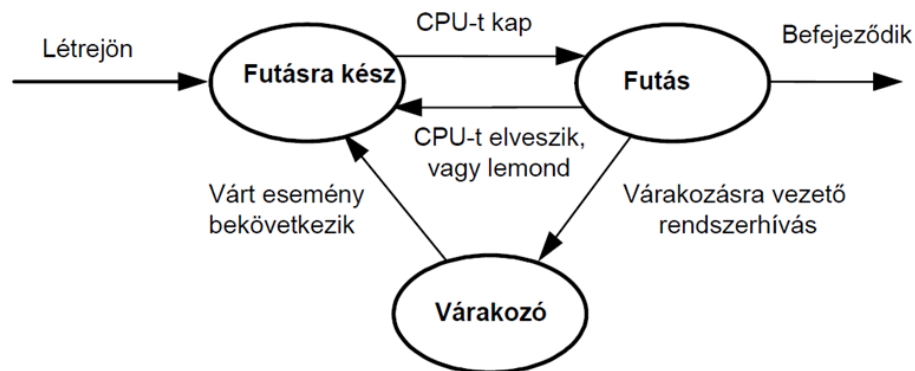
- *Futó állapot* (running). A processzé a CPU.
- *Futásra kész állapot* (ready). A CPU-t igényli a processz.
- *Blokkolt állapot* (blocked) A processz valamilyen erőforrást igényel (CPU-n kívül).

Előbb ezt kell megkapja, csak utána vehet részt a CPU-ért való versengésben.

Processz állapotok-átmenetek

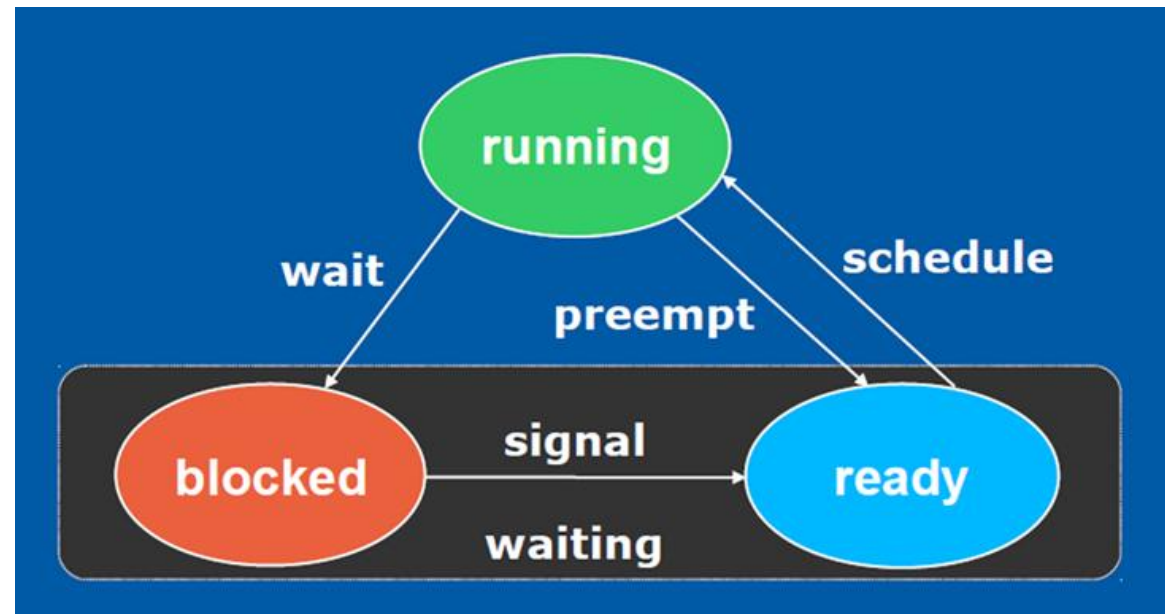
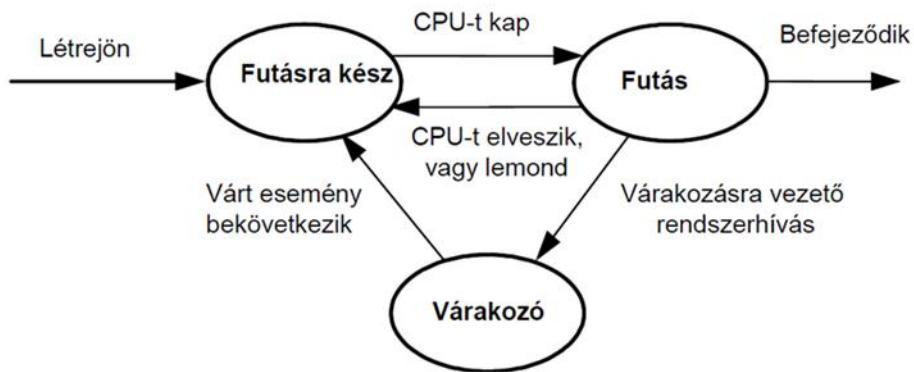
- A *wait* (sleep, request) *állapotátmenetet* maga a **processz „idéz elő”,** azzal, hogy bejelenti igényét *valamilyen erőforrásra* (pl. bufferre, diszk blokkra, eseményre stb.) és *ez nem áll azonnal a rendelkezésre.*

Ezért, hogy ne foglalja feleslegesen a CPU-t, *blokkolódik.*



Processz állapotok-átmenetek

- A *signal* állapot-átmenetet egy **IT kezelő rutin "okozza"**, jelzés, hogy *már rendelkezésre áll az erőforrás*.
- A *elvétel* (preempt) és a *kioszt állapotátmeneteket* az **ütemező (scheduler) "hajtja végre"**.



Processz állapotok – átmenetek

Alapvető processz állapotátmenetek:

- *Wait/sleep/request*: blokkolás esemény bekövetkezéséig állapotátmenet – maga a *processz kezdeményezi*.
- *Signal/respond*: jelzés küldése esemény bekövetkezésére állapotátmenet - IT okozza.
- *Preempt*: beavatkozási állapotátmenet - a CPU-t elveszi a processztől, *külső entitás váltja ki*.
- *Schedule*: kiosztó állapotátmenet - a CPU a processzhez kerül.

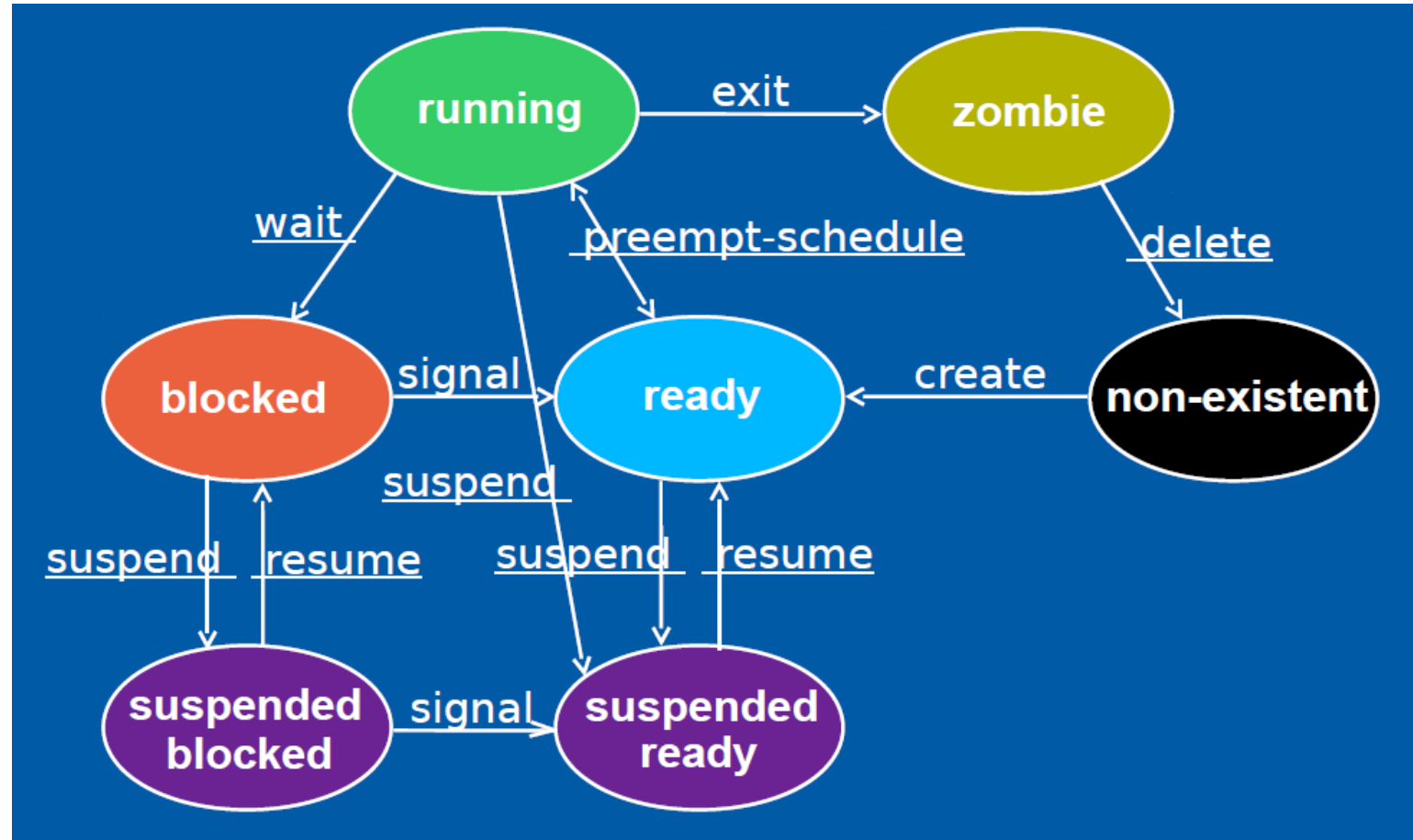
Processz rendszerhívások

- Processz *blokkoló rendszerhívások*: pause(), suspend(), sleep(), delay(), wait(), stb.
- Processz *“ébresztő” rendszerhívások*: alarm(), resume(), send-signal(), kill(), stb.
- *Statikus kontextus megváltoztatása*: execl(), execvp(), execlp(), execv(), execvp()

Processz rendszerhívások

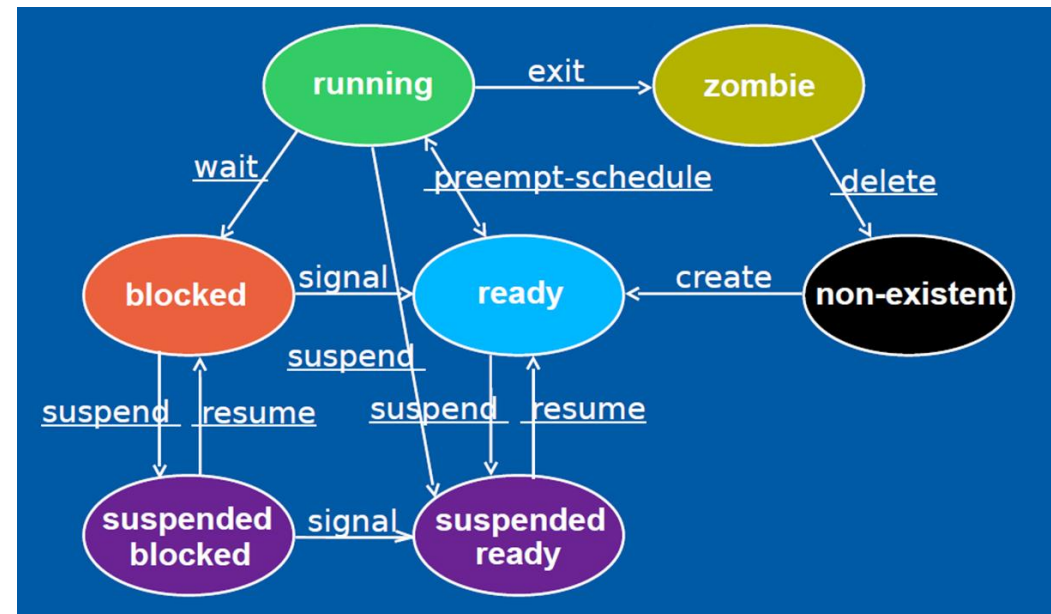
- *Processz kreáció hívásai:* `fork()`, `system()`, `clone()`, `ptrace()`
- *Processz terminációval kapcsolatos hívások:*
 - `exit()`, `_exit()`, `abort()` (a szignálkezelők is terminálhatnak)
 - `atexit()`, `on_exit()` // terminációt befolyásoló hívások
 - `wait()`, `waitpid()` // terminációt kiváró hívások

Egy bővebb - de általános - állapotdiagramm [Bach]



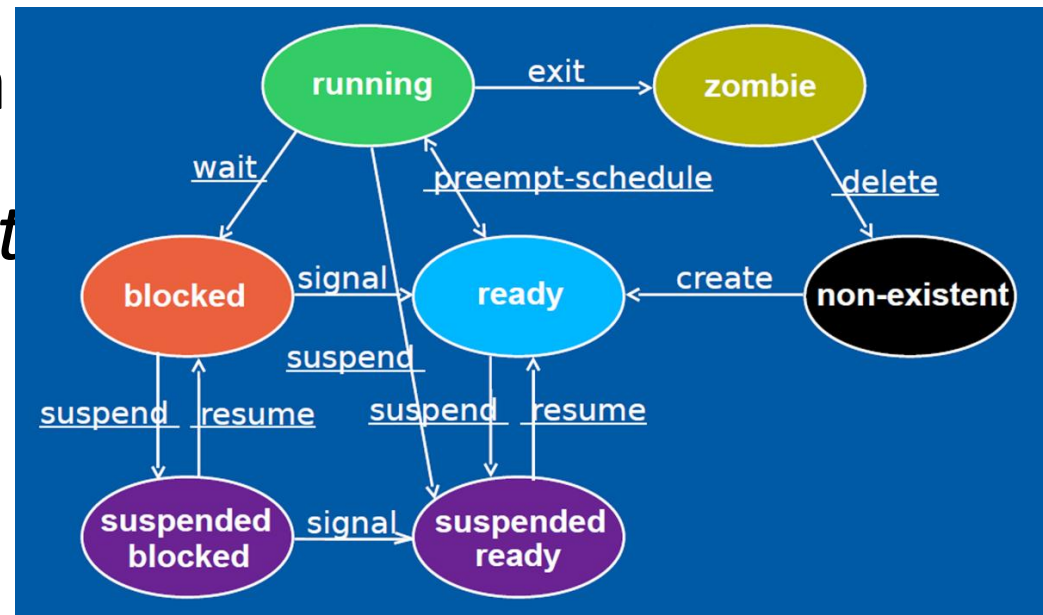
Processz állapotok

- **Zombi** állapot: a *processz már leállt*, de még nem törölődött.
- **Nem létező** állapot: a processz nem létezik.
- **Felfüggesztett** állapotok: a processz ideiglenesen ki van zárva a CPU-ért való versengésből.

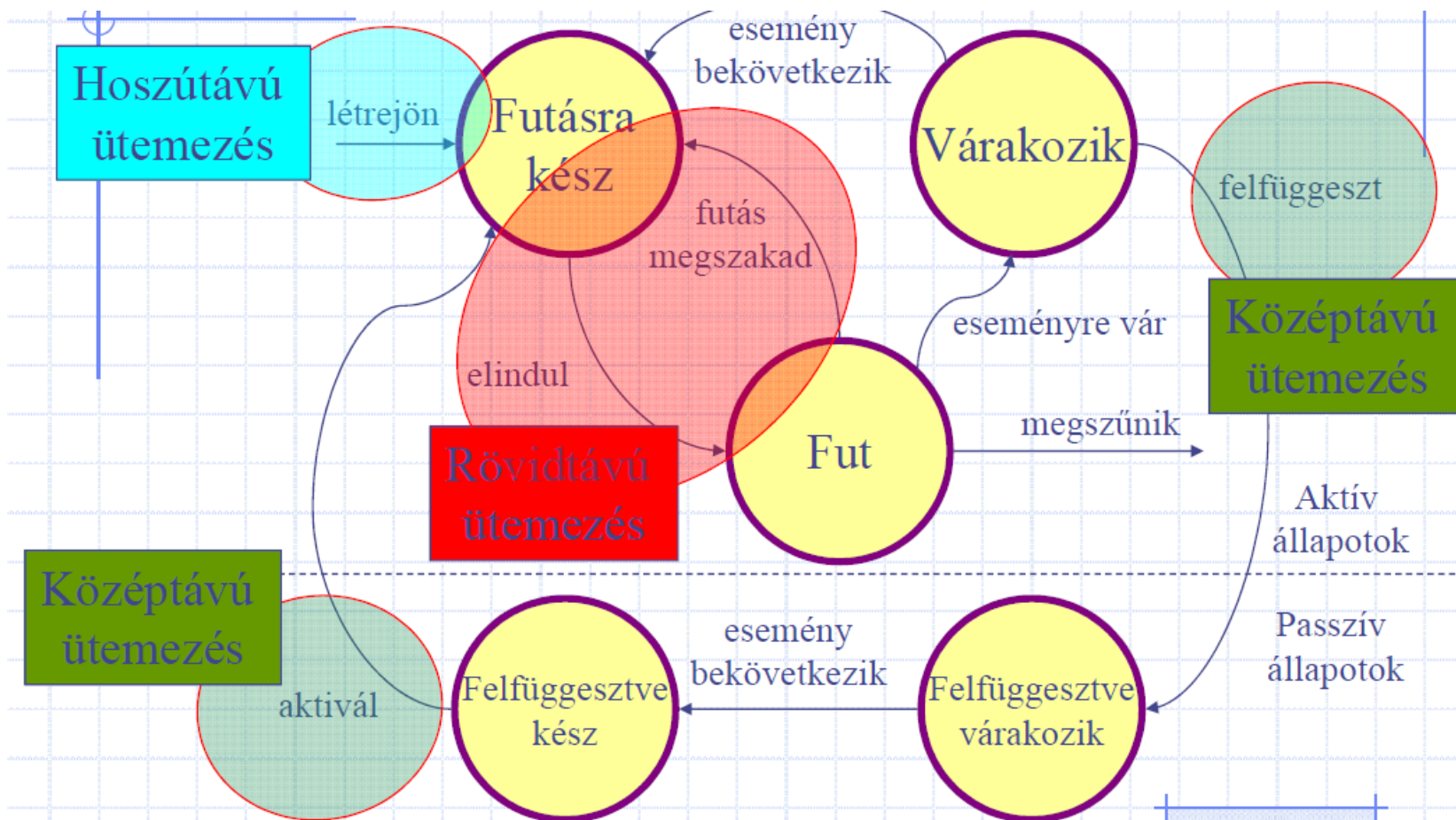


Processz állapotátmenetek

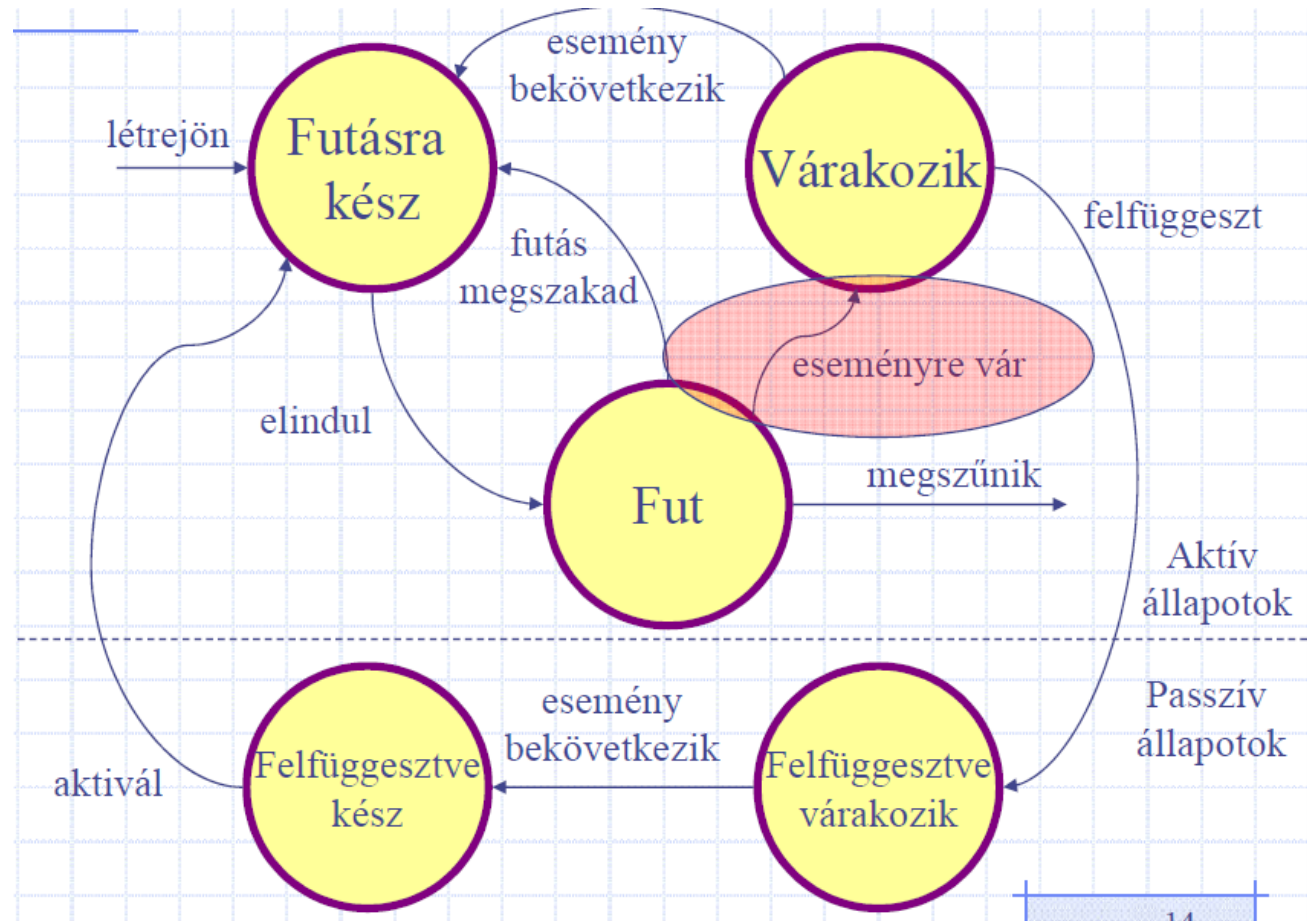
- **Exit** *állapotátmenet*: a processz terminálódott (befejeződött vagy terminálták)
- **Delete** *állapotátmenet*: a processz *törlődött, megszűnt*.
- **Create** *állapotátmenet*: a
- **Suspend, Resume** *állapotátmenet*: feladata.



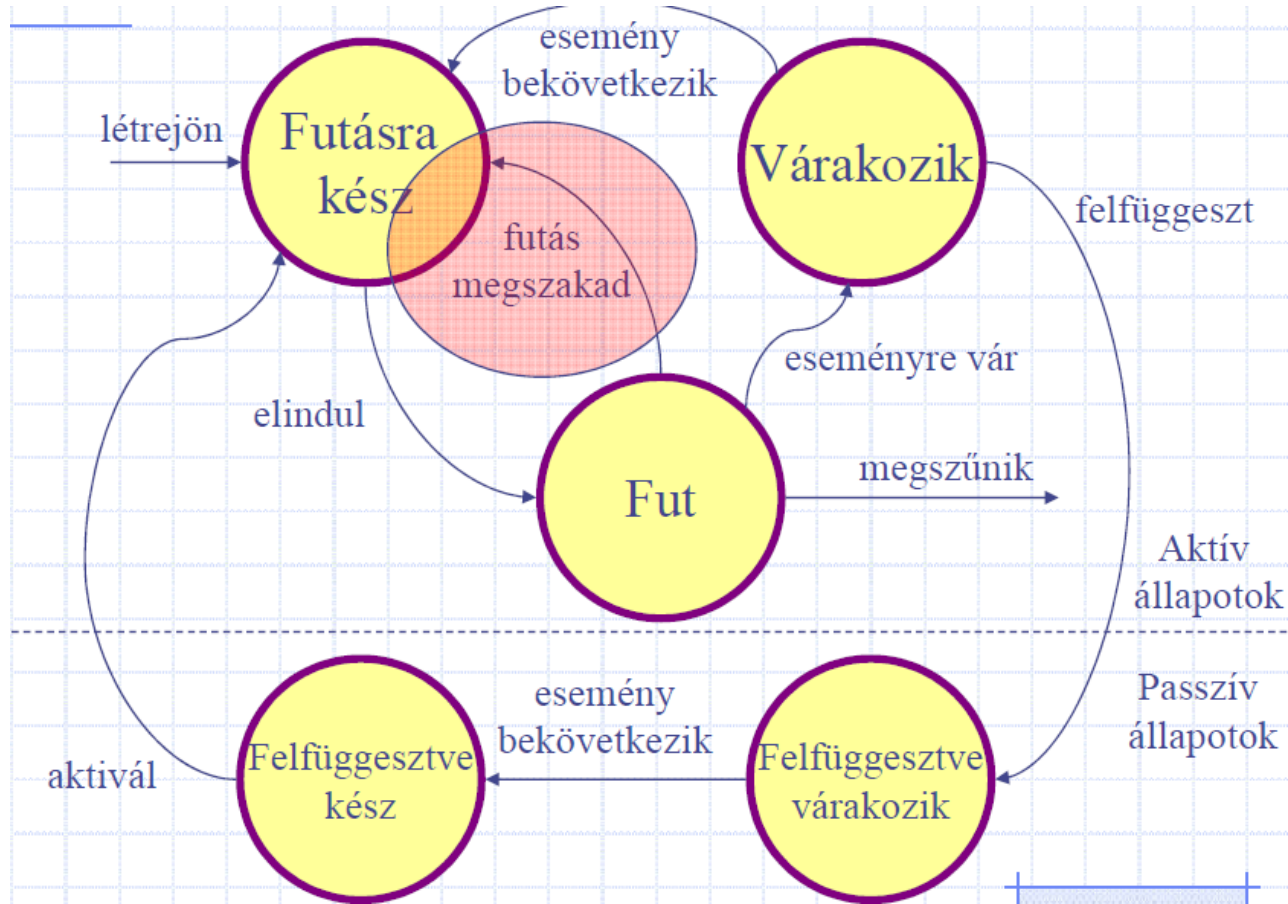
Folyamatok kibővített állapotai



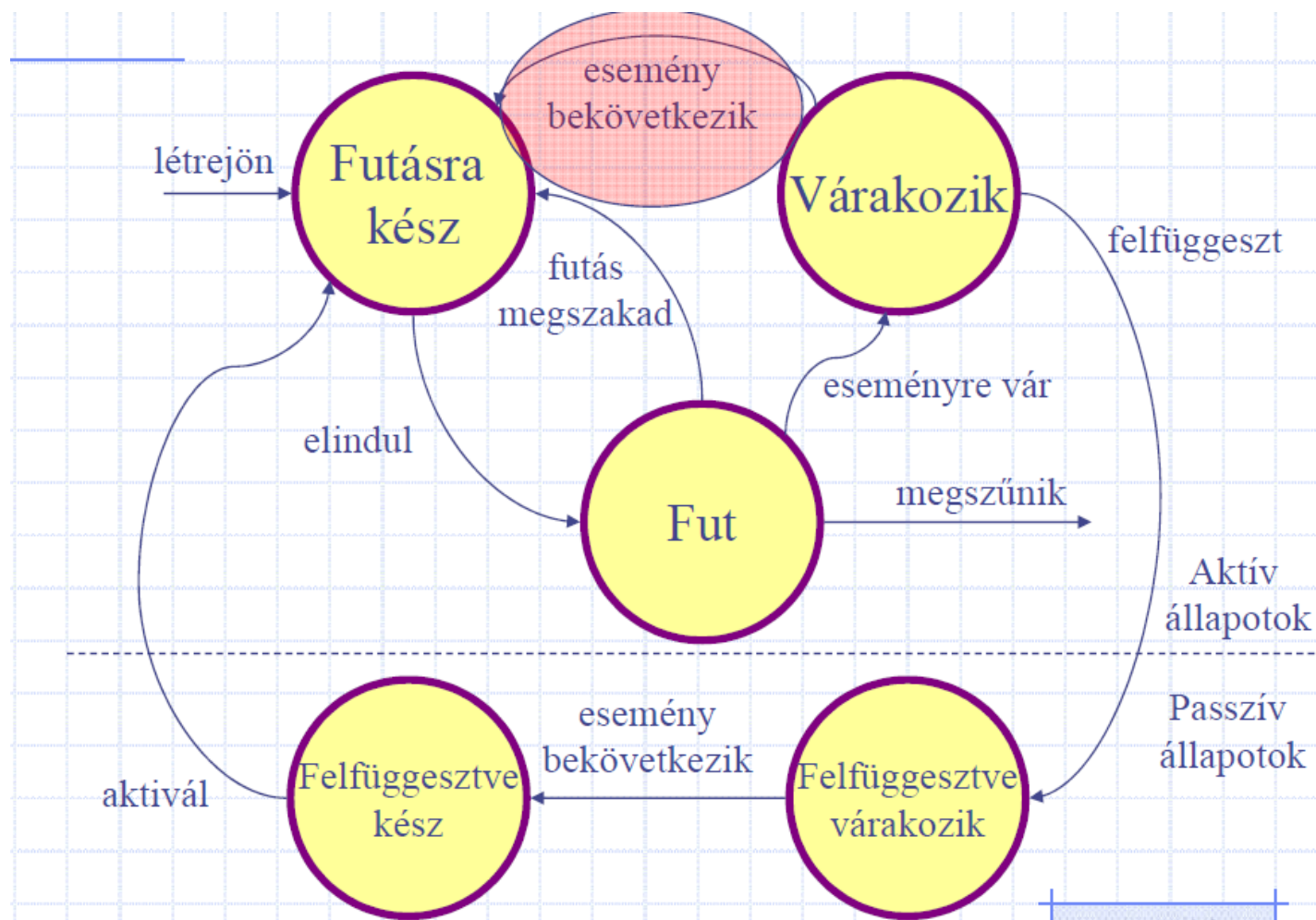
A futó folyamat várakozni kényszerül



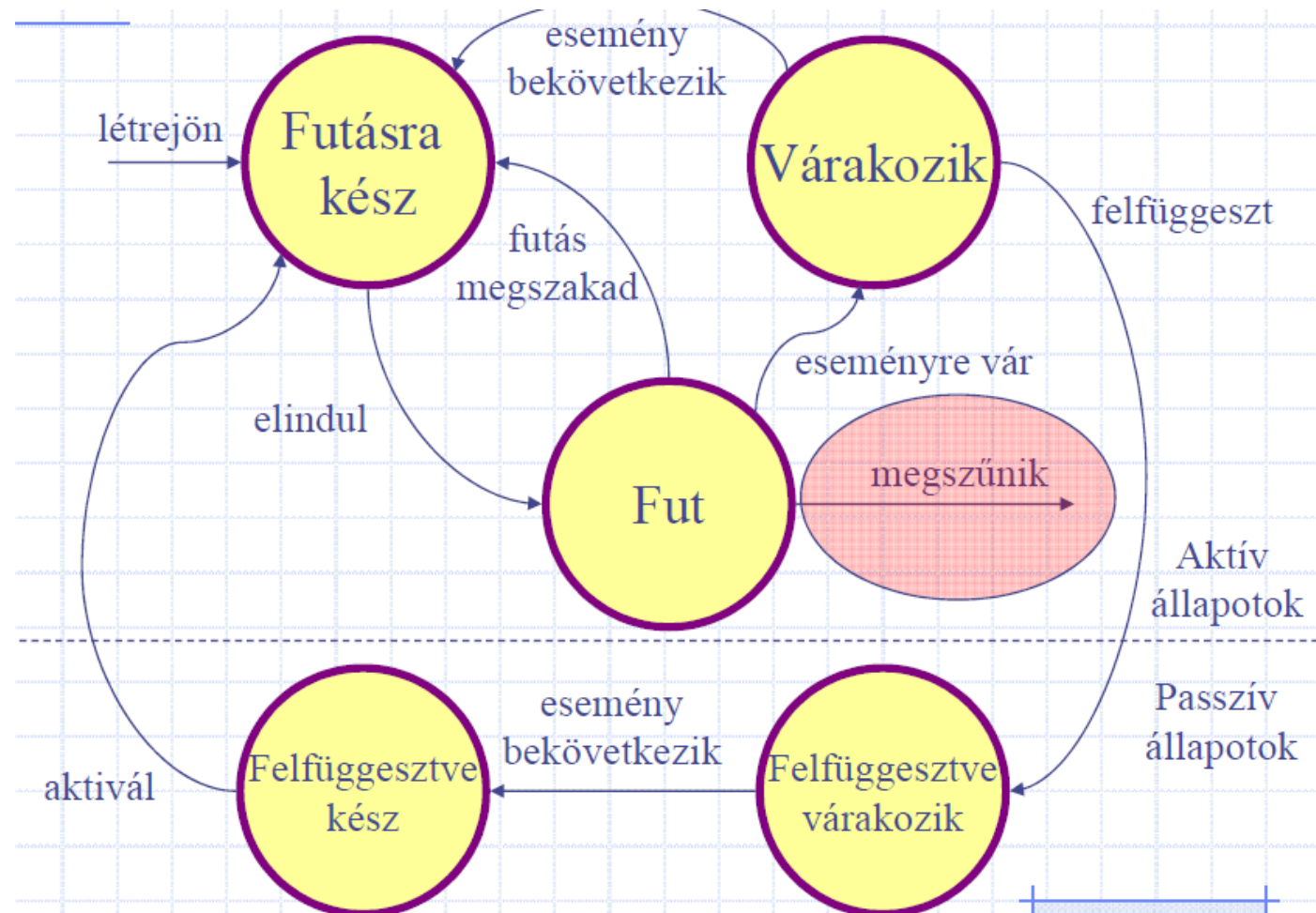
A futó folyamat lemond a CPU-ról, vagy elveszik tőle



Processz felébred, futásra készvé válik



A futó folyamat befejeződik



Processz állapotok nyilvántartása

A *PCB-n* rögzítettek az állapotok, *de láncolt lista adatszerkezeteken, sorokon* (queues) is, pl.: külön

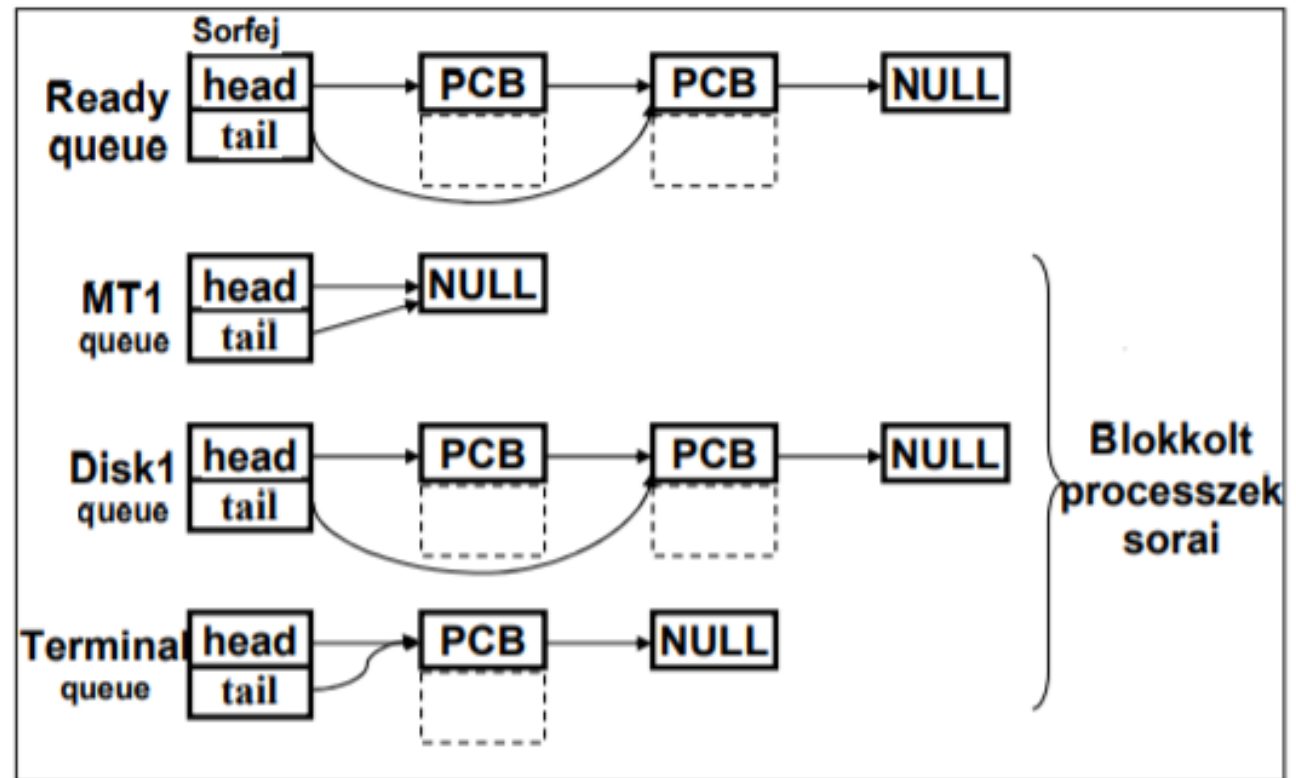
- láncolt listán vannak a *Ready processzek*,
- és az *egyes erőforrásra várok*.

Előnye: a láncolt listát *könnyebb, gyorsabb kezelnie az ütemezőnek, ill. az erőforrás kezelő rutinoknak*.

Hátránya: az állapot *változáskor több helyen kell módosítani*.

Processz állapotok nyilvántartása

Processz állapotok nyilvántartása *láncolt lista*
adatszerkezeteken,
sorokon.”[1][2][3][4][5]



1. Konzultáció -mintafeladat

a.) jegyzék létrehozása

`setxkbamp hu` – ideiglenesen hu váltás a billentyűt

`mkdir` – új mappa létrehozása

`cd ..` – egy szinttel visszalép

`rm -r` üres mappa törlése

`sudo apt install tree` – a fa installálása

b.) Másolás

`cp -r ./bokor/banan fa`

`cp -r ./land/szeder fa`

1. Konzultáció -mintafeladat

c.) áthelyezés

```
mv ./bokor/mogyoro fa
```

```
mv ./land/kokusz fa
```

d.) jegyzék törlése, fájl létrehozása

```
rm -r land
```

Belépés a jegyzékbe

```
touch leiras.txt
```

```
touch felsorolas.txt
```

1. Konzultáció -mintafeladat

e.) beírás a fájlba

```
nano leiras.txt
```

A gyümölcs finom

`Ctrl + X → Y → Enter` – kilépés a nano szövegszerkesztőből.

f.) listázza a jegyzék tartalmát almappa is megjelenjen.

```
ls -R
```

```
pwd -aktuális mappa kiírása
```

g.) összes e megjelenjen

```
find . -name ?e\* -print
```


1. Konzultáció -mintafeladat

h.) mindenki számára olvasható a leiras.txt

Belépés a banan – chmod hozzáadás az olvasási jogosultság.

```
chmod o+r leiras.txt
```

ls -a – mindent kilistáz.

ls -l – részletes információ az adott fájlról

i.) mennyi helyett foglal a neptunkod

```
du -max-depth=0
```

1. Konzultáció -mintafeladat

h.) listázás abc szerint a `leiras.txt` tartalmat

```
cat leiras.txt
```

```
sort -b leiras.txt
```

i.) számlálás

A fájlba lévő szavak számlálása:

```
wc leiras.txt
```

```
wc .w leiras.txt
```

Felhasznált irodalom

[1] Vadász Dénes: Operációs rendszerek, ME, 2007.

<http://www.iit.uni-miskolc.hu/~vadasz/GEIAL302B/2007-Ea>

[2] Kóczy, Kondorossi szerk: Operációs rendszerek mérnöki megközelítésben, Panem, 2000.

[3] Pannon Egyetem: Operációs rendszerek- ütemezés

https://dcs.uni-pannon.hu/files/docs/users/csertangyorgy/os/OS_05_Utemezes.pdf.

[4] Tannenbaum: Modern Operating Systems, Prentice Hall,
1992.

[5] Kóczy, Kondorossi szerk: Operációs rendszerek mérnöki megközelítésben, Panem, 2000.